

Final Metrics List

Class-Level Analysis

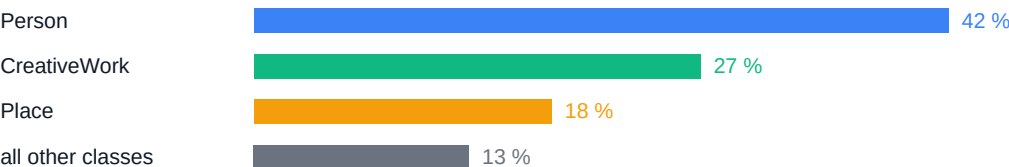
1. Class population & share

Counts the entities of every class and the fraction of the whole KG they represent. It answers the first question anyone asks of a snapshot — what is actually in this graph? — and, compared across versions, shows which classes grow, shrink or disappear over time.

$$\text{share}(t) = |E_t| / |E|$$

$|E_t|$ = number of entities of class t , $|E|$ = all entities

Share of all entities per class (one snapshot):



Across versions: watch each class's share rise or fall.

2. Property entropy per type

For one class and one predicate, the Shannon entropy of the predicate's object-value distribution. If the values are spread evenly over many objects (birthDate: nearly every person has a different one) the entropy is high and the predicate carries real information. If one value dominates (gender) the entropy is low. Tracked across versions, it shows predicates becoming more or less informative as the KG fills up.

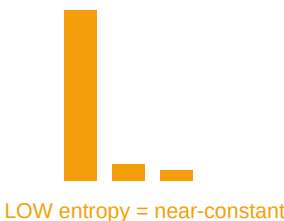
$$\text{EntF}(p, t) = - \sum_o P(o) \cdot \log_2 P(o)$$

$P(o)$ = count of value o among all values of predicate p inside class t

birthDate — values evenly spread



gender — one dominant value



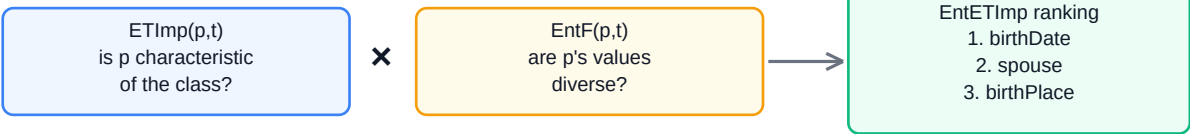
same 5 values, very different information content

3. Entropy-weighted type importance

Multiplies two signals: how characteristic a predicate is for a class (the provided ETImp baseline) and how diverse its values are (metric 2), combined as a weighted geometric mean. The result ranks, per class, the predicates that are both typical and informative.

$$\text{EntETImp}(p, t) = \text{EntF}(p, t)^w \cdot \text{ETImp}(p, t)^{1-w}$$

weighted geometric mean, $w \approx 0.75$



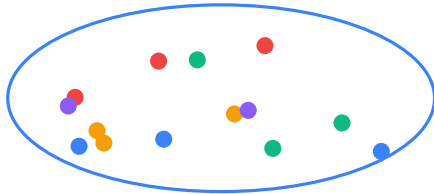
typical AND informative predicates rise to the top

4. Class entropy

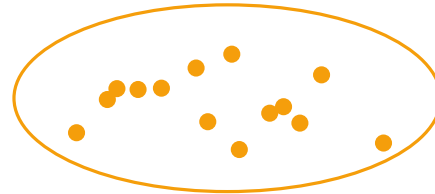
One entropy number for a whole class, over the object values and predicates of all its entities. It measures how diverse and information-rich the class is as a unit: a class whose entities all look alike scores low, a class with varied, detailed entities scores high. Its trajectory shows classes becoming richer or flatter between versions.

$$H(t) = - \sum_o P(o) \cdot \log_2 P(o)$$

over all object values o of all entities of class t



class A: varied predicates/objects — HIGH entropy



class B: everything looks the same — LOW entropy

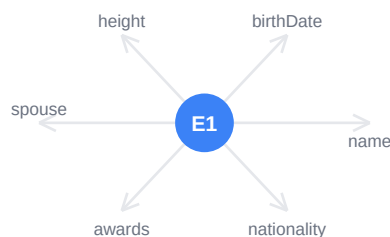
Entity-Level Analysis

5. Entity informativeness

The normalized count of an entity's datatype (literal) properties — the InfoRank idea that important things have lots of information recorded about them. It needs a single aggregation query, no iteration, and across versions it identifies entities that are being actively enriched versus abandoned.

$$IR(v) = dtp(v) / \sum_u dtp(u)$$

$dtp(v)$ = number of literal (datatype) properties of entity v



6 literal properties -> HIGH informativeness



2 literal properties -> LOW informativeness

6. Object diversity

The number of distinct objects each predicate points to, within a class or for a single entity. It separates predicates that fan out to many different values from predicates that reuse a handful, and complements the entropy metrics with a simple, exact count.

$$OD(p, t) = |\{ o : (e, p, o) \in T \text{ and } e \in E_t \}|$$

number of distinct objects of predicate p within class t

nationality → ● ● ● ● ● ● ● ● ... 195 distinct objects -> DIVERSE

gender → ● ● ● 3 distinct objects -> NARROW

count(DISTINCT object) per predicate, within a class or per entity

7. Entropy-weighted PageRank — own variant

PageRank with one change: rank flows preferentially through INFORMATIVE predicates. Every link is weighted by the property entropy of its predicate (metric 2), so an edge like birthDate — diverse, informative values — passes on more rank than a near-constant edge like gender. Knowgly's version weights links with InfoRank instead; using the per-type property entropy as the edge weight is this thesis's own, simpler variation, and it connects metric 2 directly into the importance ranking. Computed per version and per KG, the score of the same class or entity is then compared across snapshots — e.g. Person in YAGO 4 vs YAGO 4.5 vs DBpedia — revealing importance shifts over time and between graphs (classes matched across KGs by name/label, as in metric 13). Classic unweighted PageRank stays as the provided baseline.

$$PR(v) = (1 - d) + d \cdot \sum_{u \rightarrow v} w(p_{uv}) \cdot PR(u) / \text{outdeg}(u)$$

$w(p) = \text{EntF}(p) / \max_q \text{EntF}(q)$ — the (normalized) property entropy of the predicate on the edge; $d = 0.85$

birthDate
high entropy -> $w \approx 1.0$

receives rank mostly via

gender
low entropy -> $w \approx 0.1$

informative links

same power iteration,
one extra weight $w(p)$
per edge — from metric 2

Global Graph Analysis

8. Graph size & shape

The per-snapshot foundation table: triples, entities, classes, predicates, density (triples per entity) and average in-/out-degree. Computed for every version and every KG, its deltas form the growth table that makes all other comparisons interpretable.

$$\text{density} = |T| / |E|, \quad \text{avg. out-degree} = |T| / |S|$$

T = triples, E = entities, S = distinct subjects

YAGO 4 (2020)
~2.0 B triples
~64 M entities
density 31.2

Δ growth table

+25 % triples, -22 % entities:
fewer but denser entities

YAGO 4.5 (2024)
~2.5 B triples
~50 M entities
density 50.1

(numbers illustrative)

9. Class-level change rate (churn)

Localizes the change between two versions: for every class, the fraction of its triples that were added or deleted. It answers WHERE the KG evolved — whether a big diff means everything drifting slightly, or a few classes being rewritten while the rest stay stable. Computed by grouping the added and deleted triple sets of the triple-level diff (metric 10) by the subject's class, so it reuses the integer-encoded diff engine at almost no extra cost.

$$\text{churn}(t) = (|A_t| + |D_t|) / |T_t(v_1)|$$

A_t, D_t = added / deleted triples of class t between v_1 and v_2 ; $T_t(v_1)$ = its triples in the older version

Where did the change happen? churn per class (v1 -> v2)



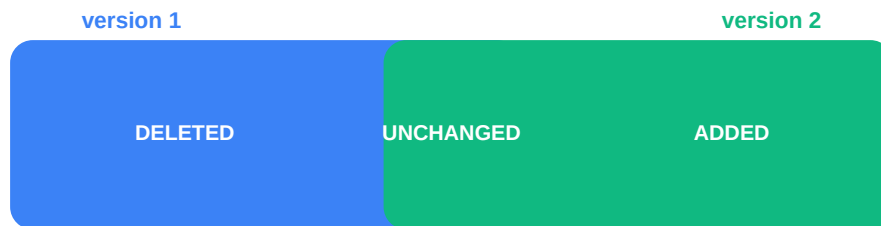
same global diff, very unevenly distributed — a few classes rewritten, the rest stable

10. Triple-level diff via integer-encoded sets

Computes the exact sets of added, deleted and unchanged triples between two versions. Every triple is first dictionary-encoded into integer IDs, so the comparison becomes a set operation over sorted integers instead of comparing strings — the efficiency contribution of the thesis, benchmarked on runtime and memory against a naive baseline.

$$\text{Added} = T_2 - T_1, \quad \text{Deleted} = T_1 - T_2, \quad \text{Unchanged} = T_1 \cap T_2$$

T_1, T_2 = integer-encoded triple sets of the two versions



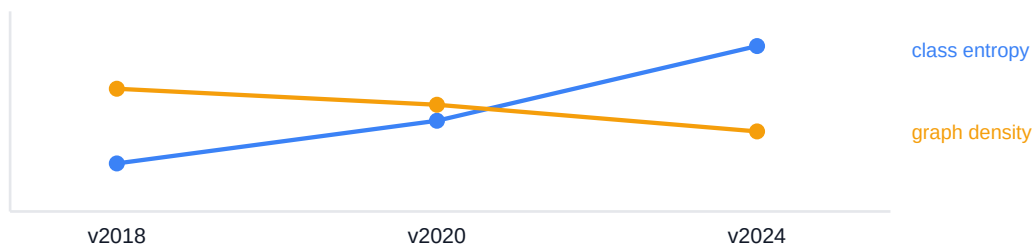
each triple (s,p,o) becomes one integer ID -> the diff is a fast set operation over sorted integers, not slow string comparison

11. Metric trajectories

The framework layer: every metric above is computed per version and per KG, and the dashboard plots the values as trajectories with deltas. This turns each per-snapshot metric into an evolution metric and is the visual-analytics backbone of the thesis.

$$\Delta m(v_i) = m(v_{i+1}) - m(v_i)$$

for every metric m and each pair of consecutive versions v_i, v_{i+1}



any per-snapshot metric, plotted over versions -> an evolution metric for free

12. Vocabulary / schema evolution (additional)

Which classes and predicates are added, removed or deprecated in each version, and how quickly new terms are actually adopted in the data. Derived almost for free from the triple-diff engine of metric 10.

$$C_{\text{added}} = C_2 - C_1, \quad C_{\text{removed}} = C_1 - C_2$$

classes C of each version; the same for the predicate sets

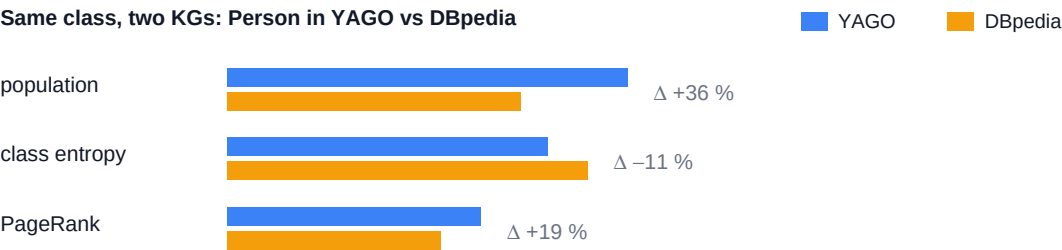


13. Cross-KG class comparison (additional)

Takes one class that exists in two different KGs — Person in YAGO and in DBpedia — and puts its metric values side by side: population, property entropy, class entropy, PageRank. The classes are matched at the schema level (by name/label); overlapping the instances of two different KGs via sameAs links is impractical, so no union or merge-gain is estimated. The comparison shows how differently the two graphs model and populate the same concept.

$$\Delta m(t) = m_A(t) - m_B(t) \text{ for every metric } m$$

same class t in the two KGs A, B — e.g. Person in YAGO vs DBpedia



classes matched by name/label at the schema level — no instance-level sameAs linkage